

beautiful_soup アプリ概要

Repo evidence のみを根拠に要約。情報が不足する項目は "Not found in repo." と明示。

What It Is

FFIII風のターン制RPGを Web/Wasm 中心に実装したアプリです。`web\_wasm` 配下ではタイトル、ロケーション、メニュー、戦闘、装備、魔法、ステータス、ジョブ画面が単一ページアプリとして構成されています。

Who It's For

明示的な一次ペルソナ: Not found in repo.

Repo evidence suggests:

FFIIIライクなRPGをブラウザで試遊・検証したい開発者 / テスター / 制作者向け。

What It Does

- タイトル画面でニューゲーム、コンテニュー、ロードを提供。
- ロケーション選択から battle / shop / inn / menu へ遷移。
- 戦闘ラウンドをブラウザ内 Pyodide 上の Python エンジンで実行。
- item / equip / magic / status / job の各サブ画面を提供。
- AUTO SAVE と Slot 1-3 の手動保存を IndexedDB に保持。
- save envelope と menu_state を localStorage / IndexedDB へ同期。
- JSON Schema によるセーブ検証と旧 save の schema 移行を実装。

How It Works

- UI: `web\_wasm/index.html` を起点に `app.js` と `router.js` が SPA を起動し、`screens/\*` を遅延読み込み。
- 状態管理: `store/app\_store.js` が route、location、menu_state、save envelope を保持。
- 保存: `shared\_storage.js` が localStorage と IndexedDB の save slot を扱う。
- 実行基盤: `pyodide\_runtime.js` が Pyodide、`typing-extension`、`jsonschema`、`bootstrap\_runtime.py` をロード。
- Python bundle: `scripts/build\_wasm\_bundle.py` が `combat` / `assets/data` / `schemas` / `system` / `utils` を `python\_bundle.zip` に固める。
- データフロー: 画面操作 → store 更新 → Pyodide 上の `combat.wasm\_api.WasmBattleEngine` / data loader 実行 → DTO / snapshot を UI に反映 → セーブ保存。
- 補助サーバ: `wasm\_app.py` は静的配信、`adapters/flask\_app.py` は Flask / Gunicorn 起点として存在。

How To Run

1. `pip install -r requirements.txt`
2. `python scripts/build\_wasm\_bundle.py`
3. `python wasm\_app.py`
4. ブラウザで `http://127.0.0.1:8000/web\_wasm/` を開く